QA Auto Bug Reporter Pro

Technical Architecture & System Design Specification

System Name: QA Auto Bug Reporter Pro	**Target Framework:** Python 3.10+ / Flask Microservice
Primary AI Model: Google Gemini 2.5 Pro Vision	**Persistence Store:** Microsoft Excel (OpenPyXL Engine)
Repository: git@github.com:nahincodezzi/qa-auto-bug-reporter.git	**Deployment Target:** Windows OS Standalone Executable / Desktop Web

1. Executive Summary & Architectural Mission

Modern enterprise software testing requires high velocity and meticulous documentation. However, Quality Assurance (QA) engineers frequently encounter operational friction when manually taking screenshots, uploading assets to cloud servers, copying URLs, and manually translating localized informal bug descriptions (Banglish, Bengali, informal notes) into standard English test cases. **QA Auto Bug Reporter Pro** is an automated end-to-end testing infrastructure designed to eliminate this friction entirely.

By integrating an asynchronous clipboard listener (Ctrl+V), automated binary cloud image streaming, Google's flagship **Gemini 2.5 Pro Multimodal Vision AI**, and an industrial-grade OpenPyXL spreadsheet automation engine, the system achieves zero-touch bug reporting directly into a persistent 15-column master corporate defect ledger (bug_report.xlsx).

2. High-Level Multi-Tier System Architecture

Tier / Layer	Core Technology	Architectural Responsibility
1. Presentation Layer	HTML5, CSS3, Vanilla ES6 JavaScript	Direct clipboard interception (Ctrl+V), multi-file thumbnail gallery, dynamic column deletion UI, real-time AI status badge.
2. Microservice Layer	Python 3.10+, Flask, Threading	RESTful endpoints (/api/add_bug, /api/delete_row, /api/delete_columns), automatic browser launch via daemon thread, file-lock handling.
3. AI & Vision Engine	Google GenAI SDK (gemini-2.5-pro)	Multimodal screenshot inspection, SQA prompt engineering, multi-model fallback cascade, zero-Banglish leakage enforcement.
4. Semantic NLP Fallback	Regex Pattern Classifier & Entity Engine	Zero-key offline operation; parses authentication, validation, forms, delete operations, and constructs 100% relevant English SQA reports.
5. Persistence Layer	OpenPyXL Spreadsheet Engine	Manages master bug_report.xlsx, automated date-based TID generation, hyperlink re-alignment, and middle/trailing sheet compaction.

3. In-Depth Component Specifications

3.1 Asynchronous Clipboard & Memory Streaming

Traditional web file upload forms require saving screenshots to the physical filesystem before browsing and uploading. The system overrides the global browser clipboard handler (window.addEventListener('paste')). When an image is intercepted from the OS clipboard, it is converted into an in-memory File Blob and rendered instantly into a responsive thumbnail gallery with an active counter badge. Uploading is executed asynchronously via multipart/form-data.

3.2 Dual-Provider Cloud Image Streaming

Screenshots are transmitted as raw byte buffers (io.BytesIO) to freeimage.host API with an automated Catbox.moe fallback. The service returns a permanent, unexpiring, publicly resolvable HTTPS direct URL (e.g. https://iili.io/ntf415X.png). When multiple screenshots are attached, the system parses and numbers each link sequentially (SS 1: ...\nSS 2: ...) and binds them into clickable hyperlink chips within both the Excel spreadsheet and the dashboard table.

3.3 Google Gemini 2.5 Pro Vision AI & Multi-Model Cascade

The application integrates Google's newly released **Gemini 2.5 Pro** flagship model. Unlike standard text-only LLMs, screenshot bytes are encoded as types.Part.from_bytes() and delivered directly to the multimodal vision pipeline. The AI visually reads the active browser URL bar, button states (e.g. loading spinners), input values, and HTTP error dialogs. A multi-model cascade guarantees zero system downtime:

- **Primary:** gemini-2.5-pro — Deepest SQA reasoning and visual precision.
- **Fallback 1:** gemini-2.5-flash — High-speed multimodal inference.
- **Fallback 2:** gemini-2.0-flash — Next-generation ultra-fast inference.
- **Fallback 3:** gemini-1.5-pro — Extended context window backup.

4. Master Excel Schema & Data Integrity Engineering

The application enforces a rigorous, standardized 15-column schema aligned with corporate enterprise SQA benchmarks:

Col #	Column Name	Type & Formatting	SQA Description
1	**TID**	String (Center / Navy Badge)	Date-based unique defect identifier (e.g. TID11601).
2	Module	String (Hierarchy)	Hierarchical module path (e.g. Super Admin > Grant Event).
3	Test Case Title	String (Bold Title)	Action-oriented verification statement starting with 'Verify...'.
4	Test Scenario	String (Pre-wrap)	Business condition or specification rule under verification.
5	Test Data	String (Pre-wrap)	Concrete test input values, parameters, or session credentials.
6	Test Step	Multi-line String (Numbered)	Sequential reproduction steps (1. ..., 2. ..., 3. ...).
7	Expected Result	String (Pre-wrap)	Exact expected business outcome according to specification.
8	Actual Result	String (Red / Pre-wrap)	Observed defect behavior detailing exact deviation from expected.
9	**Status**	Enum (Red Fill: Fail)	Test execution outcome (Fail / Pass / Blocked).
10	Screenshort	Hyperlink / Blue Underline	Public cloud URL(s) to permanent screenshot evidence.
11	Dev Comment	String (Nullable)	Developer feedback, triage notes, or fix PR references.
12	QA Comment	String (Pre-wrap)	Technical QA observations, root cause clues, or API error codes.
13	**Status**	Enum (Yellow Fill: New)	Defect lifecycle state (New / Open / In Progress / Resolved).
14	Dev	String (Center)	Assigned developer or engineering team.
15	Dev Comennt	String (Nullable)	Additional secondary engineering commentary.

5. Date-Based Dynamic TID Generation Algorithm

Test Case IDs are critical for cross-referencing between Jira, GitHub Issues, and automated test runs. The system generates TIDs using the deterministic formula: TID<Month><Day><Sequence:02d>.

- **Month & Day:** Evaluated in real-time via datetime.now() (e.g. November 6th → TID116).
- **Daily Sequential Counter:** Evaluated by inspecting existing row identifiers in Column 1. The regex pattern r'(\d+)$' extracts historical sequence numbers for today's prefix, selecting max(sequences) + 1. This guarantees sequential continuity regardless of row deletions.

6. OpenPyXL Sheet Compaction & Ghost Row Resolution

A critical technical challenge discovered in openpyxl was the **Hyperlink Coordinate Desync Bug**. When ws.delete_rows() deletes an intermediary data row, openpyxl shifts cell data upward but fails to adjust the internal ws.cell.hyperlink.ref XML attributes. Upon saving, openpyxl resurrected phantom cells at obsolete row coordinates, creating empty ghost rows displaying - | - | -. This was permanently resolved via the compact_worksheet() algorithm:

Engineering Resolution (compact_worksheet):
1. ***Iterative Coordinate Re-alignment:*** *Iterates all active cells and re-binds cell.hyperlink.ref = cell.coordinate.*
2. ***Bottom-up Trailing Purge:*** *Scans from ws.max_row backwards to Row 2, deleting fully empty trailing rows.*
3. ***Middle Ghost Row Deletion:*** *Iterates row-by-row and verifies meaningful content across columns; deletes phantom rows.*
4. ***Dynamic Header Mapping:*** *Reads active headers from Row 1, allowing safe dynamic column deletion without data corruption.*

7. Windows Concurrency & Excel File Lock Protection

When an end-user opens bug_report.xlsx in Microsoft Excel on Windows, the OS places an exclusive lock on the file descriptor. Attempts to write via openpyxl raise a PermissionError: [Errno 13]. Instead of crashing the server or hanging the terminal, the Flask controller catches PermissionError and returns a structured JSON HTTP 400 error. The UI displays an alert toast instructing the user to close Excel before retrying, ensuring flawless data safety.